

XRPL Validator — Daily Engineering Report • 2026-03-24

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **13 commits** (xrpl-validator 13)

Engineering changes

- **fix: handle mixed binary/JSON formats in ledger object decoder** (a7e2999 , xrpl-validator) RocksDB entries come in two formats: raw XRPL binary (from network RPC) and JSON (from background verification). Handle both, plus tolerance for trailing bytes in binary objects. Decode success rate: ~85% → improving as cache refreshes with clean entries.
- **fix: self-healing decoder — delete corrupted entries for re-fetch** (d9f0bf8 , xrpl-validator) Corrupted RocksDB entries (mixed JSON/binary) now get deleted on decode failure so the background fetcher re-downloads clean binary from RPC. Decode success rate: 85% → 99%+ within 2 minutes of restart.
- **feat: enable full state write-back from 33-type execution pipeline** (43f0e8a , xrpl-validator) After close_ledger applies all transactions through the complete execution pipeline (preflight → preclaim → apply_common → do_apply), write the resulting state back to RocksDB. This keeps our local state in sync with the network through full independent execution.
- **feat: incremental SHAMap + independent validation signing** (f93625e , xrpl-validator)
- SHAMap kept in memory after initial build for incremental updates - Each round: update modified entries in the full 16M-entry SHAMap - Track consecutive hash matches against network's account_hash - After 3 consecutive matches: sign validations with OUR computed hash - Log shows hash source: (src=OUR_HASH) vs (src=network) - Self-healing: corrupted entries deleted for re-fetch - Ready-to-sign status exposed via ...
- **perf: fast two-pass SHAMap builder (Pass 1: 950K entries/sec)** (cf3796e , xrpl-validator) Split SHAMap build into two passes: - Pass 1: collect all RocksDB entries into Vec (950K/s, ~20s for 18.7M) - Pass 2: bulk insert into SHAMap (slower, tree allocation bottleneck)
- **perf: compute state hash in 39s (was 15+ min) — no tree allocation** (a4d3eb8 , xrpl-validator) Replace SHAMap bulk-insert with direct bottom-up Merkle computation: - Pass 1: collect 18.7M entries from RocksDB (20s at 950K/s) - Pass 2a: compute leaf hashes (17s at 1.1M/s) - Pass 2b: recursive nibble-grouped Merkle tree (2s) - Total: 39 seconds for 18.7M entries
- **feat: state verification dashboard at /state** (7b06d7d , xrpl-validator) Visual page showing independent state hash verification: - Hero status: BUILDING → CONVERGING → VERIFIED → INDEPENDENT - Character-by-character hash comparison (green=match, red=diff) - Pulsing glow animation when hashes match - Progress bar during SHAMap build - Consecutive matches counter (3 needed to flip to OUR_HASH) - Signing source indicator (NET vs OURS) - Live ledger close log
- **feat: parallel bulk state sync (4 workers) + disable JSON write-back** (f3dd511 , xrpl-validator)

- 4 parallel workers split the key space (00-3F, 40-7F, 80-BF, C0-FF) - Each worker fetches from s1/s2 via ledger_data RPC (2048 objects/page) - ~4x faster than single-threaded (~8K objects/sec total) - Disabled JSON write-back that was corrupting binary state in RocksDB - After sync completes, SHAMap is rebuilt from fresh binary data - Progress exposed via /api/state-hash bulk_sync field
- **fix: bulk sync with rate-limit retry — 18.7M object download running** (4ea4873 , xrpl-validator)
- Fixed marker pagination: keep marker as raw serde_json::Value - Retry on rate-limit (no result field) with 5s backoff - 200ms delay between pages to avoid server overload - Debug logging to /tmp/bulk-sync.log - Disabled JSON write-back to preserve binary state integrity - Running: ~4K objects/sec, ~78 min for full 18.7M state
- **feat: 4-server parallel bulk sync — 24K objects/sec** (b5b3983 , xrpl-validator) Download state from s1.ripple.com, s2.ripple.com, xrplcluster.com, and xrpl.ws simultaneously. Each worker handles 1/4 of the key space. ~6x faster than single-server: 18.7M objects in ~13 minutes.
- **perf: 8 parallel workers across 4 servers — 36K objects/sec** (c023a8b , xrpl-validator) 2 connections each to s1.ripple.com, xrplcluster.com, xrpl.ws, xrpl.link. Each worker handles 1/8 of the key space (2 nibbles). 18.7M objects in ~9 minutes vs 78 min single-threaded.
- **fix: skip redundant bulk sync, build SHAMap from existing RocksDB** (46d7bda , xrpl-validator) Previous syncs already downloaded 18.7M objects. Build SHAMap directly from RocksDB (49s). Self-healing decoder purges corrupted JSON entries each round, background fetcher replaces with clean binary.
- **feat: s1 full-stream sync + helpers, live progress on /state page** (0d18920 , xrpl-validator)
- s1 pages through ENTIRE ledger nonstop (6K/s, no stragglers) - 6 helper workers on xrplcluster/xrpl.ws/xrpl.link fill ranges in parallel - /state page polls every 500ms showing live sync %, rate, ETA - No more waiting minutes for updates

Appendix — commits

Time	Repo	Commit	Subject
00:04	xrpl-validator	a7e2999	fix: handle mixed binary/JSON formats in ledger object decoder
00:07	xrpl-validator	d9f0bf8	fix: self-healing decoder — delete corrupted entries for re-fetch
00:11	xrpl-validator	43f0e8a	feat: enable full state write-back from 33-type execution pipeline
00:16	xrpl-validator	f93625e	feat: incremental SHAMap + independent validation signing
00:41	xrpl-validator	cf3796e	perf: fast two-pass SHAMap builder (Pass 1: 950K entries/sec)
00:46	xrpl-validator	a4d3eb8	perf: compute state hash in 39s (was 15+ min) — no tree allocation
00:49	xrpl-validator	7b06d7d	feat: state verification dashboard at /state
01:04	xrpl-validator	f3dd511	feat: parallel bulk state sync (4 workers) + disable JSON write-back
01:29	xrpl-validator	4ea4873	fix: bulk sync with rate-limit retry — 18.7M object download running
01:34	xrpl-validator	b5b3983	feat: 4-server parallel bulk sync — 24K objects/sec
01:40	xrpl-validator	c023a8b	perf: 8 parallel workers across 4 servers — 36K objects/sec
02:11	xrpl-validator	46d7bda	fix: skip redundant bulk sync, build SHAMap from existing RocksDB
02:22	xrpl-validator	0d18920	feat: s1 full-stream sync + helpers, live progress on /state page